FINTECHATHON 2026 · INTERNATIONAL TRACK · TOPIC E: SME FINANCE COPILOT

Execution evidence

A real run on the deployed NanFeng (api.nanfengcfo.com) and the same scenario in process: every step, the audit chain re-checked without the backend's code, and the Sepolia anchors.

Contents

1. 06 · Execution evidence
2. Run A · Live deploy (api.nanfengcfo.com, anchors on Sepolia)
3. Run B · In-process run (SMS OTP mode, local anchors)

Universidad del Desarrollo, Chile · Valentín Castro, Renato Lenis, Camilo Bufadel, Andy Villarroel, Vicente Rodríguez

Live app: www.nanfengcfo.com · Source: github.com/Lilvecp/nanfeng · All data is synthetic

06 · Execution evidence

Sandbox operation logs and the demo video (submission checklist item 06, for a topic that is not the on-chain one). The audit log is also anchored on the Sepolia testnet, so the evidence includes real transaction hashes.

Folder or file	What
NanFeng-Execution-Evidence.pdf	This README and both run summaries in one PDF (built with `07-team-workspace/pdf-build`)
out-live/	**Run A (live deploy)**: a real run on the deploy (https://api.nanfengcfo.com, Oct 16): the importer's PDF handed to the deployed agent and its step-by-step trace (`agent_run.json`), the owner approving the plan it proposed, one invoice registered on Sepolia and refused the second time, then the export: sandbox bank logs, audit entries with the hash-chain check, Merkle proofs and the Sepolia anchors read back. Start with `SUMMARY.md`
out/	**Run B (in-process)**: the Cherry Crisis run end to end against the backend in process (SMS OTP mode): the same files, plus `run.json` with every step
build.py	The exporter that writes both folders and re-checks the chain without the backend's code
Demo video	The live run on video, 3:45 (deliverable 03)

Export again, from the repository root:

```
cd 06-execution-evidence
../04-source-code/backend/.venv/bin/python build.py --backend ../04-source-code/backend          # in-process run ->
out/
../04-source-code/backend/.venv/bin/python build.py --url https://api.nanfengcfo.com --run-otp 482913 \
    --agent-url https://agent-production-cde7.up.railway.app --out out-live   # live deploy -> out-live/
```

`--agent-url` first sends the importer's late-payment notice (`04-source-code/web/public/samples/hongxin-payment-notice.pdf`) to the deployed agent service as the owner, and keeps the run's trace. `--run-otp` then plays the owner's side on the deploy: the agent's plan approved with the sandbox code, half of INV-2026-109 factored after its fingerprint is registered on Sepolia, the same invoice refused as already financed, and a final anchor; without it, `--url` only exports. It sends Sepolia transactions from the anchoring wallet and needs the sandbox in its opening state. Run it after a deploy that has threat-model fixes F15 and F16.

The live chain keeps growing after this run. Every visitor of the public demo shares one sandbox, so the backend resets it 30 minutes after the last activity (`04-source-code/backend/app/autoreset.py`, `NANFENG_SANDBOX_IDLE_RESET_S=1800`) and the next visitor starts at 10:13 of the crisis. The reset reseeds the bank but keeps the audit chain and its anchors (threat model F18): each reset opens a new epoch with a `sandbox.seeded` entry, so every entry in `out-live/` stays in the live chain, with the same hash, and `GET /audit/verify` keeps checking it. Every anchor and registration is also a Sepolia transaction; `anchors_on_chain.json` reads each one back. Check them on Etherscan, or follow the anchoring wallet.

The owner's notices in `out-live/` show `email · failed`. That is real: the public demo's owner is fictional (a placeholder phone and `owner@huerto-austral.example`, a reserved domain no mail server accepts) and the team's Zavu key is a test key that only reaches team phones on WhatsApp. The agent tried WhatsApp, SMS and email through the real Zavu API, every channel refused it, and the last attempt is what the trace shows. Nothing depended on it: the plan waited in Approvals, where the owner approved it. The Status column says the step ran; the Result column shows what happened to the delivery. See the technical documentation, known limitations.

All data is synthetic; no real money moves.

Run A · Live deploy (api.nanfengcfo.com, anchors on Sepolia)

Synthetic data in the NanFeng sandbox bank. Every line below is reproducible with `build.py`.

- Audit entries: **25** in the whole chain (25 in this run's sandbox, from seq 1), hash chain valid: **True**
- Anchors: **7** (evm)
  - seq 1–4 root `3e9a5ff2a43d5b79…` tx https://sepolia.etherscan.io/tx/0xfe739f65f8164295303f1312d30bfcea1827eca5f84c6cfc62ab9533caa4f77d
  - seq 5–8 root `358739d70b9be003…` tx https://sepolia.etherscan.io/tx/0x042a6327943ce418dcf74930e3c88b44198407e67624b14f59374ed72a2d7658
  - seq 9–13 root `79871fe67062eb48…` tx https://sepolia.etherscan.io/tx/0xce85b4074ac0ab736a92838ee85b3aca9ae9763963b7fc1d2812844c90be0ce9
  - seq 14–16 root `0367ad28ae134b16…` tx https://sepolia.etherscan.io/tx/0xa2296e24d4e6a6cdd3006d766c314462efc98349d94854a3e891093736de82fc
  - seq 17–17 root `d389ee5ab0d0f98b…` tx https://sepolia.etherscan.io/tx/0x1e08bf85be7624b3d137daa45aff5a5484384c1327e8736f267aa26196a38928
  - seq 18–21 root `40bf9669ec3e486c…` tx https://sepolia.etherscan.io/tx/0xcc0e57884fa205ccf706765c07ae376e21a906b9ae5d67cf5bb64dcb18c58e74
  - seq 22–24 root `bac0c5bda9ce7d0a…` tx https://sepolia.etherscan.io/tx/0x735ae05f711dd28929475544f1f3526324e5938ddfda6b0301c1577bc1a2e2da
- Merkle inclusion proofs for 5 key entries (approvals executed, attacks blocked): `proofs.json`

Checked here, without the backend's code

- Hash chain recomputed from the published formula: **25/25** entries match
- Merkle proofs recomputed from leaf to anchored root: **5/5** valid
- Anchors read back from the chain: **7/7** match
  - anchor 7: valid=True https://sepolia.etherscan.io/tx/0x735ae05f711dd28929475544f1f3526324e5938ddfda6b0301c1577bc1a2e2da
  - anchor 6: valid=True https://sepolia.etherscan.io/tx/0xcc0e57884fa205ccf706765c07ae376e21a906b9ae5d67cf5bb64dcb18c58e74
  - anchor 5: valid=True https://sepolia.etherscan.io/tx/0x1e08bf85be7624b3d137daa45aff5a5484384c1327e8736f267aa26196a38928
  - anchor 4: valid=True https://sepolia.etherscan.io/tx/0xa2296e24d4e6a6cdd3006d766c314462efc98349d94854a3e891093736de82fc
  - anchor 3: valid=True https://sepolia.etherscan.io/tx/0xce85b4074ac0ab736a92838ee85b3aca9ae9763963b7fc1d2812844c90be0ce9
  - anchor 2: valid=True https://sepolia.etherscan.io/tx/0x042a6327943ce418dcf74930e3c88b44198407e67624b14f59374ed72a2d7658
  - anchor 1: valid=True https://sepolia.etherscan.io/tx/0xfe739f65f8164295303f1312d30bfcea1827eca5f84c6cfc62ab9533caa4f77d
- Tamper test: add CLP 1 to the loan in entry 15 and the recomputed chain breaks at seq **15**
- Invoice registry (evm): **1** invoices registered against double financing: `registry.json`
  - INV-2026-109 (LEN-FACTOR:factoring) tx https://sepolia.etherscan.io/tx/0x69c7bad73fd754e8a707451ca11ebba47dfb891eebecae18144f97beed454150

Audit entries by kind

Kind	Count
`approval.approved`	3
`approval.created`	3
`approval.repeated`	2
`audit.anchored`	7

Kind	Count
`auth.login`	3
`bank.factoring`	1
`bank.loan`	1
`document.received`	2
`sandbox.seeded`	1
`security.attack_blocked`	2

The plan the owner approved is the plan the bank executed

- MILP plan (Vicente's engine): **CLP 2,106,329**
- Sandbox bank, `bank.loan` entry seq 15: **CLP 2,106,329** (interest 1,566,033 + fees 540,296)
- Runs under approval APR-0001, tier **L3** (the owner's code), executed **1** time(s): one decision per approval, even when two channels approve at once

The agent's own run

The importer's PDF went to the deployed agent service: run **RUN-0001**, status `awaiting_approval`, planner `llm`, approvals APR-0001 (L3). Every step with its tool, tier and result is in `agent_run.json`.

#	Step	Tool	Status	Result
1	Document guard	security.scan	done	clean []
2	Quarantined extraction	quarantine.extract	done	payment_notice · USD 180000.00
3	Scenario from notice	quant.scenario	done	INV-2026-118 +45 days
4	Plan	planner.llm	done	ledger_query → forecast_run → business_analysis → financing_quote → op
5	ledger_query	ledger_query	done	
6	forecast_run	forecast_run	done	81% probability of negative cash in 19 days (2026-12-30)
7	Owner notified (alert)	channel.email	done	email · failed · en
8	business_analysis	business_analysis	done	7 findings (3 high) · net 90 days CLP 451.6M · DSO 68 days
9	financing_quote	financing_quote	done	9 options, 7 eligible · need CLP 55.6M on 2026-12-30
10	optimize_plan	optimize_plan	done	Borrow USD 142,400 for 60 days from Pre-export loan in USD (PAE) · CLP
11	Owner notified (plan)	channel.email	done	email · failed · en
12	propose_plan	propose_plan	done	APR-0001 · L3 · pending · CLP 140.1M
13	anchor_audit	anchor_audit	done	Merkle root 79871fe67062… seq 9-13
14	send_collection	send_collection	done	wecom (mock) · zh · INV-2026-118
15	Owner notified (approval)	channel.email	done	email · failed · en

The three owner notices (rows 7, 11 and 15) were attempted through the real Zavu API and not delivered: the demo's owner is fictional and the team's Zavu key is a test key (why). Status says the step ran; Result shows the delivery.

Cherry Crisis run

Time	Step	HTTP	Outcome
10:12	Importer's PDF handed to the deployed agent: run RUN-0001, 15 steps, planner llm	200	awaiting_approval

Time	Step	HTTP	Outcome
10:15	Owner approves the agent's plan APR-0001 (Pre-export loan in USD (PAE)) with the one-time code	200	approved
10:20	Factoring half of INV-2026-109 proposed (L3)	201	pending
10:20	Owner approves; the invoice's fingerprint is registered on chain before the bank pays out	200	approved
10:21	Same invoice offered to another lender (double financing)	201	pending
10:21	Even with the owner's code it is refused as already financed; no money moves	200	failed: already_financed: INV-2026-109 was already financed
10:40	Owner anchors everything still pending	201	bac0c5bda9ce7d0a6fab086e7d33019a8373260cec1c4939db1ec4c436bd

Run B · In-process run (SMS OTP mode, local anchors)

Synthetic data in the NanFeng sandbox bank. Every line below is reproducible with `build.py`.

- Audit entries: **26** in the whole chain (26 in this run's sandbox, from seq 1), hash chain valid: **True**
- Anchors: **3** (local)
  - seq 1–11 root `0bf7b25358c6b3cf…`
  - seq 12–16 root `a71406c24f98c0b8…`
  - seq 17–25 root `3ecbd312549363d7…`
- Merkle inclusion proofs for 7 key entries (approvals executed, attacks blocked): `proofs.json`

Checked here, without the backend's code

- Hash chain recomputed from the published formula: **26/26** entries match
- Merkle proofs recomputed from leaf to anchored root: **7/7** valid
- Anchors read back from the chain: not applicable. This run anchors locally, with no transactions; the on-chain anchors are in Run A.
- Tamper test: add CLP 1 to the loan in entry 10 and the recomputed chain breaks at seq **10**
- Invoice registry (local): **1** invoices registered against double financing: `registry.json`

Audit entries by kind

Kind	Count
`approval.approved`	3
`approval.created`	5
`approval.otp_sent`	4
`audit.anchored`	3
`auth.login`	1
`bank.factoring`	1
`bank.loan`	1
`bank.payment`	1
`document.received`	1
`policy.kill_switch`	1
`sandbox.seeded`	1
`security.attack_blocked`	4

The plan the owner approved is the plan the bank executed

- MILP plan (Vicente's engine): **CLP 2,106,329**
- Sandbox bank, `bank.loan` entry seq 10: **CLP 2,106,329** (interest 1,566,033 + fees 540,296)
- Runs under approval APR-0001, tier **L3** (the owner's code), executed **1** time(s): one decision per approval, even when two channels approve at once

Cherry Crisis run

Time	Step	HTTP	Outcome
08:47	Supplier invoice with changed account and hidden instruction: guard	200	malicious ['doc-instruction', 'hidden-text', 'urgency-secrecy']

Time	Step	HTTP	Outcome
08:47	Same invoice: payee check against master data	200	malicious ['bank-change', 'domain-mismatch']
10:12	Payment notice from the Shanghai importer goes through the guard	200	clean
10:14	Agent proposes the MILP plan (PAE loan); the policy engine sends it to the owner (L3)	201	pending
10:14	Agent tries to approve its own plan	403	The agent cannot act as the owner
10:14	Anonymous caller tries to approve with the right code (no owner session)	401	owner_login_required
10:15	Owner signs in and approves with the one-time code received by SMS	200	approved
10:20	Agent proposes factoring half of INV-2026-109 (L3)	201	pending
10:20	Owner approves; the invoice's fingerprint is registered before the bank pays out	200	approved
10:21	Same invoice offered to another lender (double financing)	201	pending
10:21	Even with the owner's code it is refused as already financed; no money moves	200	failed: already_financed: INV-2026-109 was already financed
10:32	Agent tries to split a CLP 30M supplier payment in two	201	auto
10:32	Second half	201	pending
10:35	Owner stops the agent (kill switch on)	200	kill switch on=True
10:35	Agent tries to turn the kill switch back off	403	Only the owner can turn the kill switch off
10:35	Anonymous caller tries to turn it off	401	owner_login_required
10:40	Agent anchors everything still pending	201	3ecbd312549363d70e73780f03a5e06acf69d9979849cb96eb293b703a38